

506021334 Stats102A HW1

Bryan Mui

2025-01-16

Sourcing the R file(file should be within the working directory)

```
source("./506021334_stats102a_hw1.R")
```

Test Cases

Problem 1

Function: gcd()

implementation(using recursion):

recursively call gcd() with arguments (y, x mod y)

break recursion when y == 0,

return abs(x)

```
# Error handling test case
```

```
gcd(12.2, 14.9) # Coerce to integers
```

```
## Warning in gcd(12.2, 14.9): gcd: Inputs are not whole numbers, coercing x and y  
## using as.integer...
```

```
## [1] 2
```

```
gcd("TRUE", FALSE) # NULL
```

```
## Warning in gcd("TRUE", FALSE): gcd: Inputs are not integers/numeric, coercing x  
## and y using as.integer...
```

```
## Warning in gcd("TRUE", FALSE): NAs introduced by coercion
```

```
## Warning in gcd("TRUE", FALSE): gcd: Coercion resulted in NA values, function  
## returns NULL
```

```
## NULL
```

```
# in class test cases
```

```
gcd(48, 18L) # 6
```

```
## [1] 6
```

```
gcd(180, 25L) # 5
```

```
## [1] 5
```

```
gcd(-42112, 56399) # 7
```

```
## [1] 7
```

Function: gcdi()

Implementation:

initialize answer as first element in vector

for all elements in input vector v:

+ calculate gcd(answer, next element)

return answer

```
# Error handling test cases
```

```
gcdi(c(4.2)) # NULL
```

```
## Warning in gcdi(c(4.2)): input vector length is less than 2 or greater than
```

```
## 1000, function returns NULL
```

```
## NULL
```

```
gcdi(list("TRUE", 1, 2.6, FALSE)) # NULL
```

```
## Warning in gcdi(list("TRUE", 1, 2.6, FALSE)): input is not a vector, function
```

```
## returns NULL
```

```
## NULL
```

```
gcdi(c(2.2, 9.6, "HELLO")) # NULL
```

```
## Warning in gcdi(c(2.2, 9.6, "HELLO")): Non numeric value in vector, function
```

```
## returns NULL
```

```
## NULL
```

```
# in class test cases
```

```
gcdi(c(16, 32, -58)) # 2
```

```
## [1] 2
```

```
gcdi(c(4789, 6123, 19927334)) # 1
```

```
## [1] 1
```

Problem 2

Function: lcm()

Implementation: using the gcd() function,

return $|a * b| / \text{gcd}(a, b)$

```
# Error handling test case
lcm("HELLO", "WORLD") # NULL
```

```
## Warning in lcm("HELLO", "WORLD"): gcd: Inputs are not integers/numeric,
## coercing x and y using as.integer...
```

```
## Warning in lcm("HELLO", "WORLD"): NAs introduced by coercion
## Warning in lcm("HELLO", "WORLD"): NAs introduced by coercion
```

```
## Warning in lcm("HELLO", "WORLD"): gcd: Coercion resulted in NA values, function
## returns NULL
```

```
## NULL
```

```
lcm(148.9, 3452.123456) # 127,724, coerces by removing the decimal(rounding down)
```

```
## Warning in lcm(148.9, 3452.123456): gcd: Inputs are not whole numbers, coercing
## x and y using as.integer...
```

```
## [1] 127724
```

```
# in class test cases
lcm(4, 6) # 12
```

```
## [1] 12
```

```
lcm(123, 4920) # 4920
```

```
## [1] 4920
```

```
lcm(8675309, 9876543) # ~8.5682062377 * 10^13
```

```
## [1] 8.568206e+13
```

Function: add_2_frac() Implementation: calculate denominator using lcm(d1, d2)
calculate numerator using the formula: $((\text{denom} / d1) * n1) + ((\text{denom} / d2) * n2)$
return list, where num=numerator and denom=denominator

```
# Error handling test case
add_2_frac(1.1, FALSE, 1, 3) # NaN
```

```
## Warning in add_2_frac(1.1, FALSE, 1, 3): add_2_frac: Inputs are not
## integers/numeric, coercing n1, d1, n2, d2 using as.integer...
```

```
## $num
## [1] NaN
##
## $denom
## [1] 0
```

```
# in class test cases
add_2_frac(1, 2, 1, 3) # 5/6
```

```
## $num
## [1] 5
##
## $denom
## [1] 6
```

```
add_2_frac(1, 11, 1, 5) # list(num = 16, denom = 55)
```

```
## $num
## [1] 16
##
## $denom
## [1] 55
```

```
add_2_frac(1, 11, -1, 5) # list(num = -6, denom = 55)
```

```
## $num
## [1] -6
##
## $denom
## [1] 55
```

```
add_2_frac(2468, 3579, 1357, 2468) # 10947727 / 8832972
```

```
## $num
## [1] 10947727
##
## $denom
## [1] 8832972
```

Problem 3

Function: is_prime() Implementation:
initialize logical vector of length x as FALSE
for all values in vector x:

- if number is less than or equal to 1, set num as not prime
- if number is 2 or 3, set num as prime
- if number is greater than or equal to 4 AND even, set as not prime
- else if floor(sqrt(x)) is less than 3, number is prime
- for all odd numbers $\leq \text{floor}(\sqrt{x})$:
 - if any $x \bmod (\text{odd number})$ is 0, set num as not prime

return answer as logical vector

```
# Error handling test case
is_prime(c(TRUE, "FALSE", NA, 48, 1.2, 3L)) # NULL
```

```
## Warning in is_prime(c(TRUE, "FALSE", NA, 48, 1.2, 3L)): Non numeric value in
## vector, function returns NULL
```

```
## NULL
```

```
# in class test cases
is_prime(c(9, 7)) # [False True]
```

```
## [1] FALSE TRUE
```

```
is_prime(c(1, 3, 5)) # [FALSE TRUE TRUE]
```

```
## [1] FALSE TRUE TRUE
```

```
is_prime(c(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)) # [FALSE TRUE TRUE FALSE TRUE FALSE TRUE FALSE FALSE FALSE]
```

```
## [1] FALSE TRUE TRUE FALSE TRUE FALSE TRUE FALSE FALSE FALSE
```

```
is_prime(c(29, 97, 143, 169, 221, 323, 377, 493, 541, 599)) # [TRUE TRUE FALSE FALSE FALSE FALSE FALSE]
```

```
## [1] TRUE TRUE FALSE FALSE FALSE FALSE FALSE FALSE TRUE TRUE
```

Function: get_factors()

Implementation:

initialize answer as list(primes, exponents)

initialize copy of x

initialize increment starting at 2

while x is greater than 1:

- if x mod incr == 0:
 - divide x by incr
 - add prime if incr not in prime, otherwise add 1 to exponent for that prime
- else:
 - add 1 to incr repeatedly until incr is prime(using is_prime())

return answer as list

```
# Error handling test case
get_factors(1.92) # appropriate error handling: NULL
```

```
## Warning in get_factors(1.92): get_factors: Inputs are not whole numbers,
## coercing x using as.integer...
```

```
## $primes
## NULL
##
## $exponents
## NULL
```

```
get_factors(377.777) # primes = [13 29] exponents [1 1]
```

```
## Warning in get_factors(377.777): get_factors: Inputs are not whole numbers,
## coercing x using as.integer...
```

```
## $primes
## [1] 13 29
##
## $exponents
## [1] 1 1
```

```
# in class test cases
x <- sample(x = 1e4, size = 1)
y <- get_factors(x)
this_works <- prod(y$primes^y$exponents) == x & all(is_prime(y$primes))
if (this_works) {
  print("passed test case!")
} else {print("false!")}
```

```
## [1] "passed test case!"
```

```
# other test cases
get_factors(1920) # list(primes = [2 3 5], exponents = [7 1 1])
```

```
## $primes
## [1] 2 3 5
##
## $exponents
## [1] 7 1 1
```

```
print(get_factors(1532512)) # list(primes = [2 83 577], exponents = [5 1 1])
```

```
## $primes
## [1] 2 83 577
##
## $exponents
## [1] 5 1 1
```